COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	08-03-26	DATE SUBMITTED:	08-05-26
NAME:	Christian Lee C. Bandigan	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Hands-on Activity 4.1 Stacks

1. PROCEDURE OUTPUT

ILOA

HOA4 > ILOA > STLstack.cpp > main()

```cpp
#include <iostream>
#include <stack>

int main(){
    std::stack<int> newStack;
    std::stack<int> newStack2;

    newStack.push(3); //Adds 3 to the stack
    newStack.push(8);
    newStack.push(15);

    //returns a boolean response depending on if the stack is empty or not
    std::cout << "Stack Size: " << newStack.empty() << std::endl;

    //returns the size  of the stack itself
    std::cout << "Stack Size: " << newStack.size() << std::endl;

    //returns the topmost element of the stack
    std::cout << "Top Element of the Stack: " << newStack.top() << std::endl;

    //removes the top most element of the stack
    newStack.pop();

    std::cout << "Top Element of the Stack: " << newStack.top() << std::endl;

    std::cout << "Stack Size: " << newStack.size() << std::endl;

    //emplace
    newStack.emplace(20);
    std::cout << "The element at the top of the stack is: "<<newStack.top()<<std::endl;
    std::cout << "The size of the stack is: " << newStack.size()<< std::endl;

    //swap
    std::cout<< "is stack2 empty? "<<newStack2.empty()<<std::endl;
    std::cout<<"Swapping stack1 and stack2"<<std::endl;
    newStack.swap(newStack2);
    std::cout<< "is stack2 empty? "<<newStack2.empty()<<std::endl;
    std::cout << "The element at the top of stack2 is: "<<newStack2.top()<<std::endl;
    std::cout<< "is stack1 empty? "<<newStack.empty()<<std::endl;

    return 0;
}
```

Figure 1. Code for ILOA

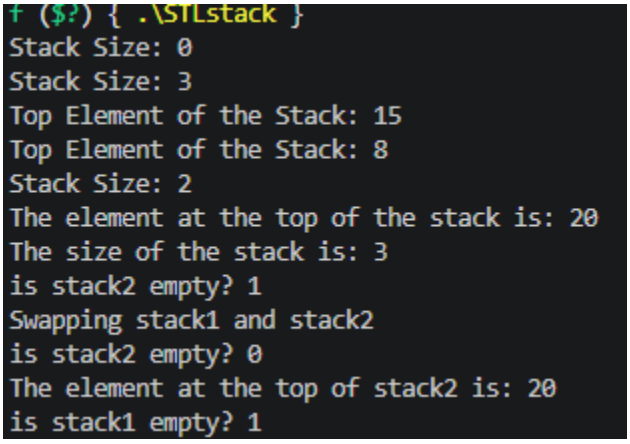


Figure 2. Output for ILOA

Observations

The stack library consists of seven methods that either transform or observe stacks. Size tells how many elements are in the stack, top tells what element is at the top of the stack, and isempty returns a boolean value if the stack is empty or not. Push can add a new element at the top of the stack. Pop removes the element at the top of the stack. Swap from the name itself swaps the contents of one stack, with another. Lastly, emplace also adds an element at the top of the stack but constructs a new element at the top instead of copying or moving like push.

ILOB.1

stacks > stack_custom > Bandigan_Seatwork4_1.2.cpp > displayStack()

```cpp
#include <iostream>

//global declarations
#define maxCap 10
int stackArr[maxCap];
int top = -1, newData;

//prototype functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void displayStack();

int main(){
//main driver
int choice;
    while(true){
        std::cout << "================================================================"<< std::endl;
        std::cout << "Stack Operations \n";
        std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6.displayStack 7. exit" <<std::endl;
        std::cout << "================================================================"<< std::endl;
        std::cin>>choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<<"is the stack empty? " << isEmpty()<<std::endl;
            break;
            case 5: std::cout <<"is the stack full? " << isFull()<<std::endl;
            break;
            case 6: displayStack();
            break;
            case 7: return 0;
            break;
            default: std::cout<< "Invalid Choide" << std::endl;
            break;
        }
    }
    return 0;
}
```

Figure 3.

```
48   //function definition
49   bool isEmpty(){
50       if(top == -1) return true;
51       return false;
52   }
53
54   bool isFull(){
55       if(top == maxCap - 1) return true;
56       return false;
57   }
```

Figure 4. Check Stack Functions

Description

These functions just check the stack if they are empty or full. In the case of array stacks it just uses the top and maxCap variables because they are used as indexes in the array.

```
59   void push(){
60       //error checking
61       if(isFull()){
62           std::cout << "Stack overflow"<<std::endl;
63           return;
64       }
65       //pushing the stack
66       std::cout << "Enter  a new value: " <<std:: endl;
67       std::cin >> newData;
68       //how do we insert the data into the stack?
69       stackArr[++top] = newData;
70   }
```

Figure 5. Push Function

Description

The push function adds a new element in the stack by just first incrementing the top variable and assigning the user input to the array using top as the index.

```
72   void pop(){
73       if(isEmpty()){
74           std::cout << "Stack underflow" <<std::endl;
75           return;
76       }
77
78       //display the value that we are going to pop
79       std::cout << "Popping: " << stackArr[top] << std::endl;
80       //decrement the top value from the stack
81       top--;
82   }
```

Figure 6. Pop function

Description

The pop function removes an element in the stack by decrementing the top variable.

```
83  void Top(){
84      if(isEmpty()){
85          std::cout << "The stack is empty"<<std::endl;
86          return;
87      }
88      std::cout<<"The value at the top of the stack: "<<stackArr[top] << std::endl;
89  }
```

Figure 7. Top function

Description

The top function prints the element at the top of the stack by using the top variable in the array.

```
90  void displayStack(){
91      int temp[maxCap];
92      int tempv = -1;
93      if(isEmpty()){
94          std::cout << "The stack is empty" <<std::endl;
95          return;
96      }
97      std::cout << "Contents of stack" << std::endl;
98      while(top != -1){
99          temp[++tempv] = stackArr[top];
100         std::cout << stackArr[top] << " ";
101         top--;
102     }
103     while(tempv != -1){
104         stackArr[++top] = temp[tempv];
105         tempv--;
106     }
107     std::cout<<std::endl;
108 }
```

Figure 8. Display Stack Function

Description

To display all the contents of the stack I first created a temporary array and top variable to hold the content of the stack later. I then created a while loop that inserts the top variable into the temporary array, prints the top element, and pops the top element. After that another while loop was created to return the elements in the stack array.

```
==========================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6.displayStack 7. exit
==========================================================
1
Enter  a new value:
10
==========================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6.displayStack 7. exit
==========================================================
1
Enter  a new value:
9
==========================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6.displayStack 7. exit
==========================================================
1
Enter  a new value:
8
==========================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6.displayStack 7. exit
==========================================================
6
Contents of stack
8 9 10
```

Figure 9. Output for ILOB.1

ILOB.2

HOA4 > ILOb > ilob2.cpp > ...

```cpp
#include <iostream>
#include "stackList.h"

int main(){
    push(1);
    std::cout << "After the first PUSH top of the stack is: ";
    Top<int>();

    push(5);
    std::cout << "After the second PUSH top of the stack is: ";
    Top<int>();

    displayAll<int>();

    pop<int>();
    std::cout << "After the first POP operation, the top of stack is: ";
    Top<int>();

    pop<int>();
    std::cout << "After the second POP operation, the top of stack is: ";
    Top<int>();

    pop<int>();

    return 0;
}
```

Figure 10. Driver for ILOB.2

HOA4 > ILOb > stackList.h > displayAll<T>()

```cpp
#ifndef STACK_LIST_H
#define SRTACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
        T data;
        Node* next;
};

//global pointer for the top of the  stack
template <typename T>
Node<T> *head = nullptr;

```

Figure 11. Node Class and Global Pointer

```
18  //push
19  template <typename T>
20  void push(T newData){
21      //create a new node and assign the newData
22      Node<T>* newNode = new Node<T>;
23      newNode ->data = newData;
24
25      //new node points to current top, then becomes the new top
26      newNode->next = head<T>;
27      head<T> = newNode;
28  }
```

Figure 12. Push Function for linked list

Description

The push function for linked list stack just creates a new node, assigns the data, points its next pointer to the current head and then becomes the new head.

```
30  //pop
31  template <typename T>
32  T pop(){
33      //check of the stack is empty
34      if(head<T> == nullptr){
35          std::cout << "stack underflow." << std::endl;
36      }
37
38      //save the current top node
39      Node<T>* temp = head<T>;
40      //store its data
41      T tempVal = temp->data;
42
43      //move the head to the next node
44      head<T> = head<T>->next;
45      //free the removed node
46      delete temp;
47      //return the removed value
48      return tempVal;
49  }
```

Figure 13. Pop Function for linked list

Description

The pop function removes the top element for linked list stack by creating a temp pointer to save the current top node, creating a temp variable to store its data, moving the head to the next node, deleting the temp node, and returning the removed value.

```
51  //isEmpty
52  template <typename T>
53  bool isEmpty(){
54      return head<T> == nullptr;
55  }
56
57  //top
58  template <typename T>
59  void Top(){
60      //error catching:
61      if (isEmpty<T>()){
62          std::cout << "Stack underflow" <<std::endl;
63          return;
64      }
65      //display the Top node
66      std::cout << "Top of Stack: " << head<T>->data << std::endl;
67  }
```

Figure 14. Check Function for linked list

Description

These functions check the top of the stack or if the stack is empty by checking the head node if it is null or has data.

```
69  //display All
70  template <typename T>
71  void displayAll(){
72      if (isEmpty<T>()){
73          std::cout << "Stack underflow" <<std::endl;
74          return;
75      }
76      Node<T>* temp = head<T>;
77      std::cout << "Contents of stack:" << std::endl;
78      while(temp != nullptr){
79          std::cout << temp->data;
80          if(temp->next != nullptr){
81              std::cout << " -> ";
82          }
83          temp = temp->next;
84      }
85      std::cout <<std::endl;
86  }
87
88  #endif
```

Figure 15. Display Stack Function for linked list

Description

Displaying the whole stack in a linked list implementation is done by creating a temporary pointer to the head node and iterating the whole list through that pointer using a while loop.

```
After the first PUSH top of the stack is: Top of Stack: 1
After the second PUSH top of the stack is: Top of Stack: 5
Contents of stack:
5 -> 1
After the first POP operation, the top of stack is: Top of Stack: 1
After the second POP operation, the top of stack is: Stack underflow
stack underflow.
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA4\ILOb>
```

Figure 16. Output for ILOB.2

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

A. Stack using Arrays

HOA4 > ILOC > ILOC_arrayy > supp.cpp > main()

```cpp
#include <iostream>
#include <string>

#define maxCap 100
char stackArr[maxCap];
int top = -1;
std::string newData;

void push(char symbol);
void pop();
bool match(char open,char close);

int main(){
    std::cout << "Enter Expression:" << std::endl;
    std::getline(std::cin, newData);

    for(char c: newData){
        if(c == '(' || c == '[' || c== '{'){
            push(c);
        }else if (c == ')' || c == ']' || c== '}'){
            if(top == -1){
                std::cout <<"Error: Stack is empty." << std::endl;
                break;
            }else{
                if(match(stackArr[top],c)){
                    pop();
                }else{
                    std::cout <<"Error: Symbol not matched." << std::endl;
                }
            }
        }else{
            continue;
        }
    }
    if(top != -1){
        std::cout <<"Missing Closing Symbol." << std::endl;
    }else{
        std::cout << "Expression has balanced Symbols"<<std::endl;
    }

    return 0;
}
```

```
44  void push(char symbol){
45      if(top == maxCap - 1){
46          std::cout << "Stack Overflow" << std::endl;
47          return;
48      }
49      stackArr[++top] = symbol;
50  }
51
52  void pop(){
53      if(top == -1){
54          std::cout << "Stack Underflow" << std::endl;
55          return;
56      }
57      top--;
58  }
59
60  bool match(char open, char close){
61      if(open == '(' && close == ')' || open == '[' && close == ']' || open == '{' && close == '}'){
62          return true;
63      }else return false;
64  }
```

Figure 17.Code for ILOC using Arrays

Analysis

To solve this problem using array stack I first created the global declarations needed for the implementation. I then used the push and pop functions from ILOB. I created a bool function that returns true if its first argument is equal to the second. In the main function I used getline to get the whole string user input and assigned it to the string variable. I then used a range based for loop to traverse the string. Inside the for loop, there is an if-elif-else statement. If the current character is an opening symbol it will use the push function to add it to the stack. If it is a closing symbol it checks the stack is currently empty. If it is it will return an error else it will go through another if else statement, if the current head in the stack is equal to the closing symbol it will pop the stack else an error will occur. Else it will just continue the for loop. After the for loop is done, if the stack is not empty it will return another error because this means one opening symbol is missing a closing symbol else the given expression is balanced.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Enter Expression: (A+B)+(C-D) Expression has balanced Symbols	It outputs the correct Answer.
((A+B)+(C-D)	N	Enter Expression: ((A+B)+(C-D) Missing Closing Symbol.	It outputs the correct Answer.
((A+B)+[C-D])	Y	Enter Expression: ((A+B)+[C-D]) Expression has balanced Symbols PS C:\Users\User\OneDrive\Documents\C	It outputs the correct Answer.
((A+B]+[C-D]}	N	Enter Expression: ((A+B]+[C-D]} Error: Symbol not matched. Error: Symbol not matched. Missing Closing Symbol.	It outputs the correct Answer.

B. Stack using Linked list

HOA4 > ILOC > ILOC_list > ILOC_list.cpp > ...

```cpp
#include <iostream>
#include <string>
#include "stackList.h"

bool match(char open,char close);

int main(){
    std::string newData;
    std::cout << "Enter Expression:" << std::endl;
    std::getline(std::cin, newData);

    for(char c: newData){
        if(c == '(' || c == '[' || c== '{'){
            push(c);
        }else if (c == ')' || c == ']' || c== '}'){
            if(isEmpty<char>()){
                std::cout <<"Error: Stack is empty." << std::endl;
                break;
            }else{
                if(match(head<char>->data,c)){
                    pop<char>();
                }else{
                    std::cout <<"Error: Symbol not matched." << std::endl;
                }
            }
        }else{
            continue;
        }
    }
    if(!isEmpty<char>()){
        std::cout <<"Missing Closing Symbol." << std::endl;
    }else{
        std::cout << "Expression has balanced Symbols"<<std::endl;
    }

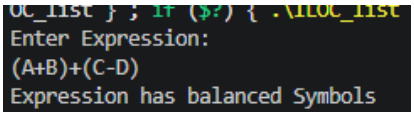
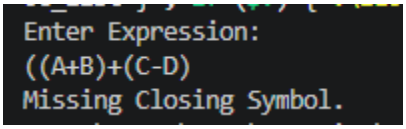
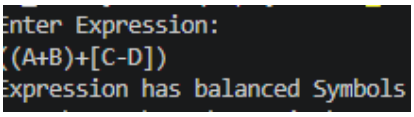
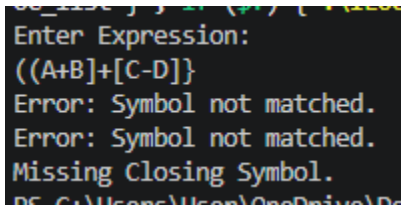
    return 0;
}

bool match(char open, char close){
    if(open == '(' && close == ')' || open == '[' && close == ']' || open == '{' && close == '}'){
        return true;
    }else return false;
}
```

Figure 18. Code for ILOC using Linked List

Analysis

To solve this problem using linked list stack I first included the header file I used for the ILOB.2 to use the push, pop, and isEmpty functions. I created a bool function that returns true if its first argument is equal to the second. In the main function I used getline to get the whole string user input and assigned it to the string variable. I then used a range based for loop to traverse the string. Inside the for loop, there is an if-elif-else statement. If the current character is an opening symbol it will use the push function to add it to the stack. If it is a closing symbol it checks the stack is currently empty. If it is it will return an error else it will go through another if else statement, if the current head in the stack is equal to the closing symbol it will pop the stack else an error will occur. Else it will just continue the for loop. After the for loop is done, if the stack is not empty it will return another error because this means one opening symbol is missing a closing symbol else the given expression is balanced.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		It outputs the correct Answer.
((A+B)+(C-D)	N		It outputs the correct Answer.
((A+B)+[C-D])	Y		It outputs the correct Answer.
((A+B]+[C-D]}	N		It outputs the correct Answer.

C. Stack using C++ STL

HOA4 > ILOC > ILOC_STL > ILOC_STL.cpp > ...

```cpp
#include <iostream>
#include <string>
#include <stack>

bool match(char open,char close);

int main(){
    std::stack<char> stack1;
    std::string newData;
    std::cout << "Enter Expression:" << std::endl;
    std::getline(std::cin, newData);

    for(char c: newData){
        if(c == '(' || c == '[' || c== '{'){
            stack1.push(c);
        }else if (c == ')' || c == ']' || c== '}'){
            if(stack1.empty()){
                std::cout <<"Error: Stack is empty." << std::endl;
                break;
            }else{
                if(match(stack1.top(),c)){
                    stack1.pop();
                }else{
                    std::cout <<"Error: Symbol not matched." << std::endl;
                }
            }
        }else{
            continue;
        }
    }
    if(!stack1.empty()){
        std::cout <<"Missing Closing Symbol." << std::endl;
    }else{
        std::cout << "Expression has balanced Symbols"<<std::endl;
    }

    return 0;
}

bool match(char open, char close){
    if(open == '(' && close == ')' || open == '[' && close == ']' || open == '{' && close == '}'){
        return true;
    }else return false;
}
```

Figure 19. Code for ILOC using STL

Analysis

To solve this problem using STL stack I first included the stack library to use the push, pop, and empty functions. I created a bool function that returns true if its first argument is equal to the second. In the main function I used getline to get the whole string user input and assigned it to the string variable. I then used a range based for loop to traverse the string. Inside the for loop, there is an if-elif-else statement. If the current character is an opening symbol it will use the push function to add it to the stack. If it is a closing symbol it checks the stack is currently empty. If it is it will return an error else it will go through another if else statement, if the current head in the stack is equal to the closing symbol it will pop the stack else an error will occur. Else it will just continue the for loop. After the for loop is done, if the stack is not empty it will return another error because this means one opening symbol is missing a closing symbol else the given expression is balanced.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Enter Expression: (A+B)+(C-D) Expression has balanced Symbols	It outputs the correct Answer.
((A+B)+(C-D)	N	STL } ; if ($?) { .\ILOC_STL Enter Expression: ((A+B)+(C-D) Missing Closing Symbol.	It outputs the correct Answer.
((A+B)+[C-D])	Y	Enter Expression: ((A+B)+[C-D]) Expression has balanced Symbols	It outputs the correct Answer.
((A+B]+[C-D]}	N	Enter Expression: ((A+B]+[C-D]} Error: Symbol not matched. Error: Symbol not matched. Missing Closing Symbol.	It outputs the correct Answer.

Conclusion

In this activity what a stack is and how to implement it in different ways. Stack is a linear data structure which follows LIFO(last in first out). I was able to use my prior knowledge to implement stack using arrays, list, and the STL. Moreover, I was able to create a function that will check if a given expression has balanced symbols using stacks. In conclusion, I was able to create a stack using the C++ STL, develop C++ code that uses both arrays and linked lists to create a stack, and solve problems using three implementations of stack.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.